Functions

Reusable, Testable, Pythonic Code

■ LEARNING OUTCOME

By the end of this module you will write clean, documented Python functions with all parameter types, use decorators, understand scope and closures, write lambda functions, and apply functional programming patterns.

■ Skills You Will Gain
• Write functions with positional, keyword, and default params
• Use *args and **kwargs for flexible signatures
• Understand local/global/nonlocal scope
• Create closures and decorators
• Write and use lambda functions
• Apply map(), filter(), functools.reduce()

■ **Functions are the foundation of reusable code. Well-designed functions -- single responsibility, clear names, type hints, docstrings -- are what separate novice from professional Python.**

SECTION 1

Function Basics

```
# Function definition def greet(name): """Return a greeting string for the given name. Args: name (str): The person's name Returns: str: Greeting message """ return f"Hello, {name}!" print(greet("Alice")) # Hello, Alice! # Default parameters def create_user(name, role="viewer", active=True, limit=100): return {"name": name, "role": role, "active": active, "limit": limit} create_user("Alice") # uses all defaults create_user("Bob", role="admin") # override role only create_user("Carol", active=False) # keyword argument # *args -- variable positional arguments (tuple) def sum_all(*numbers): return sum(numbers) sum_all(1, 2, 3, 4, 5) # 15 # **kwargs -- variable keyword arguments (dict) def show_info(**data): for key, value in data.items(): print(f"{key}: {value}") show_info(name="Alice", city="Mumbai", age=28) # Combined: positional, *args, keyword-only, **kwargs def mixed(pos1, pos2, *args, keyword_only=True, **kwargs): print(f"pos: {pos1},{pos2}, args: {args}") print(f"kw_only: {keyword_only}, kwargs: {kwargs}") mixed(1, 2, 3, 4, keyword_only=False, extra="data") # Type hints (Python 3.5+) -- not enforced but great for documentation def calculate_bmi(weight_kg: float, height_m: float) -> float: """Calculate BMI. Returns float rounded to 2 places.""" return round(weight_kg / height_m**2, 2) # Multiple return values (actually returns tuple) def minmax(numbers: list) -> tuple[float, float]: return min(numbers), max(numbers) lo, hi = minmax([3,1,4,1,5,9,2,6]) print(f"Min: {lo}, Max: {hi}")
```

SECTION 2

Scope, Closures and Decorators

```
# SCOPE -- LEGB rule: Local -> Enclosing -> Global -> Built-in x = "global" def outer(): x = "enclosing" def inner(): x = "local" print(x) # local inner() print(x) # enclosing outer() print(x) # global # global and nonlocal keywords counter = 0 def increment(): global counter # modify global variable counter += 1 def make_counter(): count = 0 def inc(): nonlocal count # modify enclosing scope variable count += 1 return count return inc c = make_counter() print(c()) # 1 print(c()) # 2 print(c()) # 3 -- count persists via closure! # CLOSURE -- inner function remembers outer scope def multiplier(factor): def multiply(number): # closes over factor return number * factor return multiply double = multiplier(2) triple = multiplier(3) print(double(10)) # 20 print(triple(10)) # 30 # DECORATORS -- functions that wrap other functions import time import functools def timer(func): @functools.wraps(func) # preserve original function metadata def wrapper(*args, **kwargs): start = time.perf_counter() result = func(*args, **kwargs) elapsed = time.perf_counter() - start print(f"{func.__name__} took {elapsed:.4f}s") return result return wrapper @timer # same as: slow_function = timer(slow_function) def slow_function(n): return sum(range(n)) slow_function(1_000_000) # Decorator with arguments def retry(max_attempts=3, delay=1.0): def decorator(func): @functools.wraps(func) def wrapper(*args, **kwargs): for attempt in range(max_attempts): try: return func(*args, **kwargs) except Exception as e: if attempt == max_attempts - 1: raise time.sleep(delay) return wrapper return decorator @retry(max_attempts=5, delay=0.5) def fetch_data(url): pass # network call that might fail
```

SECTION 3

Lambda and Functional Programming

```
# Lambda -- anonymous single-expression function double = lambda x: x * 2 add = lambda x, y: x + y clamp = lambda x, lo, hi: max(lo, min(hi, x)) # Lambda common uses people = [{"name":"Alice","age":28},{"name":"Bob","age":35},{"name":"Carol","age":22}] people.sort(key=lambda p: p["age"]) # sort by age sorted_by_name = sorted(people, key=lambda p: p["name"]) # map() -- apply function to every element (returns iterator) numbers = [1,2,3,4,5] squared = list(map(lambda x: x**2, numbers)) # [1,4,9,16,25] strings = list(map(str, numbers)) # ["1","2","3","4","5"] # filter() -- keep elements where function returns True evens = list(filter(lambda x: x%2==0, numbers)) # [2,4] # functools.reduce() -- fold sequence to single value from functools import reduce product = reduce(lambda acc, x: acc * x, numbers) # 120 running = reduce(lambda acc, x: acc + [acc[-1]+x], numbers[1:], [numbers[0]]) # Prefer comprehensions over
```

```
map/filter in Python squared = [x**2 for x in numbers] # cleaner than map evens = [x for x in numbers if x%2==0] # cleaner than filter # functools.partial -- freeze some arguments from functools import partial power = lambda base, exp: base ** exp square = partial(power, exp=2) # fix exp=2 cube = partial(power, exp=3) print(square(5)) # 25 print(cube(3)) # 27
```

■ PRACTICE Q & A

Q1. What is the LEGB rule in Python?

A: *LEGB = Local, Enclosing, Global, Built-in. When Python looks up a name, it searches in this order: first the local scope (inside the function), then any enclosing function scopes, then the global module scope, then Python's built-in names.*

Q2. What is a closure?

A: *A function that remembers variables from its enclosing scope even after the enclosing function has returned. make_counter() returns inc() which still has access to count. Closures create private state without classes.*

Q3. What does @functools.wraps do in a decorator?

A: *Copies the original function's __name__, __doc__, and other attributes to the wrapper. Without it, the decorated function appears to be named 'wrapper' which breaks debugging and introspection.*

Q4. What is the difference between *args and **kwargs?

A: **args collects extra positional arguments into a tuple. **kwargs collects extra keyword arguments into a dict. def fn(*args, **kwargs) accepts any number of any arguments.*

Q5. When should you use lambda instead of def?

A: *Lambda for short, single-expression functions used inline (sort keys, callbacks). Use def for anything needing: a docstring, multiple statements, or reuse by name. Readability wins -- if lambda is hard to read, use def.*

■ Functions Cheat Sheet	
`def fn(a, b=10, *args, **kwargs):`	Full parameter syntax
`-> return_type`	Type hint for return value
`"""docstring"""`	Document your function
`return x, y`	Return multiple values as tuple
`global x`	Modify global variable inside fn
`nonlocal x`	Modify enclosing scope variable
`lambda x: x*2`	Anonymous single-expression fn
`@decorator`	Apply decorator
`@functools.wraps(fn)`	Preserve metadata in decorator
`map(fn, iterable)`	Apply fn to every item
`filter(fn, iterable)`	Keep items where fn is True
`partial(fn, arg=val)`	Pre-fill some arguments